SEMINÁRIO · GRUPO DE PESQUISA SERG · PPGCC/UFPI · 2026

Agentic Harness Engineering

Observability-Driven Automatic Evolution of Coding-Agent Harnesses

Lin, Liu, Pan, Lin, Dou, Xi, Huang, Yan, Han, Gui & Jiang
Fudan University · Peking University · Shanghai Qiji Zhifeng Co., Ltd
arXiv:2604.25850v4 [cs.CL], 18 de maio de 2026

Apresentação:

- **Diego Cordeiro de Oliveira**

PPGCC UFPI PROGRAMA DE PÓS-GRADUAÇÃO EM CIÊNCIA DA COMPUTAÇÃO

2 Roteiro da apresentação

1. **Contexto e problema**
   O que é o harness, por que automatizar e a pergunta central
2. **Método: NexAU, Agent Debugger, Evolve Agent e o laço**
   Os três pilares de observabilidade e o Algoritmo 1
3. **Experimentos: RQ1, RQ2 e RQ3**
   Desempenho, transferência, ablação e autoatribuição
4. **Análise e conclusões**
   Limitações, leitura crítica e contribuições
5. **Referências**
   Material de apoio para a discussão

Em uma frase

O artigo trata o **harness** como objeto de otimização automática: um agente edita o harness de outro agente de código, e cada edição vem com uma **previsão verificável** que a rodada seguinte confirma ou reverte.

Preprint arXiv · Terminal-Bench 2 · SWE-bench-verified · Código público

3 Contexto

O que é o harness

Tudo o que fica **fora do modelo** e o cerca:

- **system prompt** (configuração comportamental e restritiva da IA)
- **ferramentas** e seus contratos
- **middleware** (contexto, execução, recuperação)
- **skills**, **subagentes** e **memória**

Coding agents em tarefas longas (SWE-bench, Terminal-Bench) dependem do **modelo** e do **harness**.

Por que automatizar

O harness ideal é específico do modelo. Um harness ajustado a um modelo pode ir pior em outro.

Hoje o ajuste é manual: inspecionar trajetórias, achar padrões de falha e editar prompts, ferramentas e middleware à mão.

Modelos evoluem rápido, o ajuste manual não acompanha.

Fonte: Lin et al. (2026), Resumo e Seção 1.

4 Pergunta central

PERGUNTA DE PESQUISA

Como um agente pode evoluir **todos os componentes editáveis** do harness, de forma **conjunta e estável**?

OBSTÁCULO	RESPOSTA DO AHE
Espaço de ação heterogêneo Prompt, ferramentas, middleware e memória têm formatos e efeitos diferentes.	→ **1 Observabilidade de componentes**
Trajetórias volumosas Milhões de tokens brutos enterram o sinal.	→ **2 Observabilidade de experiência**
Edições difíceis de atribuir Sem contrato, não se sabe qual edição causou qual efeito.	→ **3 Observabilidade de decisão**

Tese dos autores: o gargalo é a observabilidade, não a capacidade do agente.

Fonte: Lin et al. (2026), Resumo e Seção 1.

5 Visão geral do AHE



1 **Componentes: arquivos editáveis**

2 **Experiência: evidência destilada**

3 **Decisão: previsão verificável**

Modelo-base fixo: só o harness explícito é editado. Os três agentes (código, debugger, evolução) usam o mesmo modelo.

Fonte: recriação a partir da Figura 2 de Lin et al. (2026).

6 Pilar 1 · NexAU: componentes como arquivos

1. **System prompt**
   regras e disciplina
2. **Descrição de ferramenta**
   contrato exposto ao modelo
3. **Implementação de ferramenta**
   código executado
4. **Middleware**
   hooks antes e depois
5. **Skills**
   procedimentos reutilizáveis
6. **Config. de subagente**
   delegação
7. **Memória de longo prazo**
   lições persistentes

1 edição lógica = 1 commit
diff e rollback por arquivo

Observabilidade de componentes

- Cada padrão de falha mapeia para **uma classe de componente**.
- Acoplamento fraco: adicionar middleware não exige mexer no prompt.

Semente NexAU$_0$ (H_0)

- Só uma ferramenta de shell.
- Sem middleware, skills ou subagentes.
- Semente mínima: cada componente adicionado precisa **provar seu valor**.

Fonte: Lin et al. (2026), Seção 3.1; Nex-AGI (2025); Nex-AGI Team (2025).

7 Pilar 2 · Agent Debugger: experiência como evidência

Traços brutos
~10 milhões de tokens · k rollouts por tarefa

↓

Relatório por tarefa
causa raiz da falha ou padrão do sucesso + status pass/fail

↓

Visão geral do benchmark
~10 mil tokens · ponto de entrada de cada iteração

~1.000×
menos tokens no ponto de entrada (10M → 10K)

Divulgação progressiva

Cada mensagem de trajetória vive em seu próprio arquivo, navegável por shell e scripts.

Detalhe sob demanda

Os traços originais (brutos e levemente limpos) continuam disponíveis para **conferir as afirmações** dos relatórios.

Fonte: Lin et al. (2026), Seção 3.2; Agent Debugger: Lin (2026).

8 Pilar 3 · Evolve Agent: decisões verificáveis

Controlabilidade

- Escreve **só no workspace** do harness.
- Somente leitura: runs, tracer, verificador e configuração do LLM.
- Prompt semente não pode ser apagado.

Baseada em evidência

- Toda edição é acompanhada de uma **entrada no manifesto**.
- A rodada seguinte compara previsto e observado e **reverte o que falhou**.

Anatomia de uma entrada do manifesto

Evidência da falha
o que o relatório mostrou

→

Causa raiz inferida
por que falhou

→

Correção direcionada
qual componente muda

→

Impacto previsto
correções esperadas + regressões em risco

Impacto previsto = tarefas que devem passar + tarefas em risco de piorar. Substitui a justificativa por um contrato mensurável.

Fonte: Lin et al. (2026), Seção 3.3 e Apêndice B.2.

9 Algoritmo do AHE: seis fases por iteração

1 Rollout
k execuções por tarefa com o harness atual

2 Limpeza
normaliza os traços para a forma canônica

3 Atribuição
t ≥ 2: verifica o manifesto anterior e faz rollback do que falhou

4 Destilação
Agent Debugger gera a evidência em camadas

5 Evolução
Evolve Agent edita o workspace e escreve novo manifesto

6 Commit
marca a iteração no git; atualiza o melhor harness

A atribuição vem antes da destilação: o veredito de cada edição já entra no corpus de evidência da rodada.

Fonte: Lin et al. (2026), Algoritmo 1 (Seção 3.3).

10 Configuração experimental

Item	Configuração
Benchmark de evolução	Terminal-Bench 2: 89 tarefas (4 fáceis, 55 médias, 30 difíceis), timeout de 1 h
Transferência	SWE-bench-verified: 500 tarefas em 7 repositórios
Modelo (3 papéis)	GPT-5.4, raciocínio high (código, debugger e evolução)
Modelos alternativos	GPT-5.4 medium e xhigh, qwen-3.6-plus, gemini-3.1-flash-lite-preview, deepseek-v4-flash
Campanha	1 execução: T = 10 iterações, k = 2 rollouts por tarefa, cerca de 32 h
Métricas	pass@1 e tokens/trial; falhas de infraestrutura contam como falha
Baselines	Humanos: OpenCode, Terminus-2, Codex. Autoevolução: ACE e TF-GRPO (mesma semente NexAU_0)

Perguntas de pesquisa

1. **RQ1 · Desempenho**
   Por que engenharia agêntica, e não harnesses humanos ou outros métodos automáticos?

2. **RQ2 · Overfitting**
   O harness transfere para outro benchmark e outros modelos?

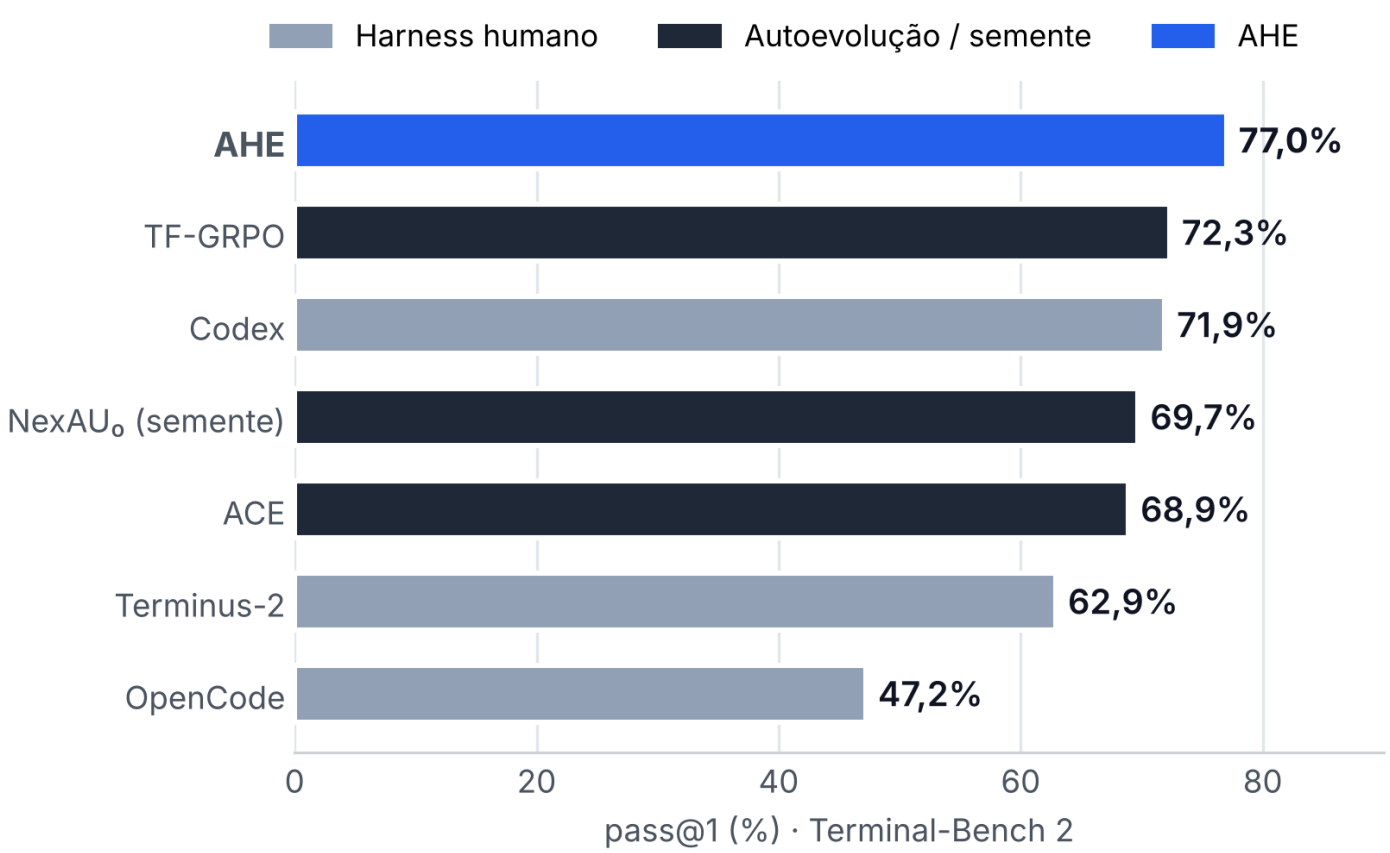
3. **RQ3 · Mecanismo**
   Onde está o ganho e quão confiável é a autoatribuição?

Fonte: Lin et al. (2026), Seção 4.1 e Apêndice A (Tabela 4).

11 RQ1: o AHE lidera no Terminal-Bench 2

PERGUNTA (RQ1)

Por que engenharia agêntica de harness, e não harnesses projetados por humanos ou outros métodos automáticos?



+7,3 pp
sobre a semente (69,7% → 77,0%)

+5,1 pp
sobre o Codex (71,9%)

Análise

- ACE (68,9%) ficou **abaixo da própria semente**.
- Único ponto fraco: nível **Difícil**, em que o Codex tem 56,7% e o AHE 53,3%.

Fonte: Lin et al. (2026), Tabela 1 (pass@1, 89 tarefas).

12 RQ2 (1/2): transferência para o SWE-bench-verified

PERGUNTA (RQ2)

O AHE sofre overfitting ao alvo da otimização? Aqui: transfere para **tarefas não vistas** (SWE-bench-verified)?

Método	Sucesso (500 tarefas)	Tokens por trial (mil)
ACE	74,6%	679
TF-GRPO	74,2%	582
$NexAU_0$ (semente)	75,2%	526
AHE	**75,6%**	**461**

+0,4 pp
sucesso sobre a semente

−12%
tokens vs. semente (−32% vs. ACE, −21% vs. TF-GRPO)

- Harness **congelado**, evoluído só no Terminal-Bench 2 (sem reevolução no SWE-bench).
- ACE e TF-GRPO ficam abaixo da semente e gastam **11% a 29% mais tokens**: o texto destilado acompanha cada chamada.

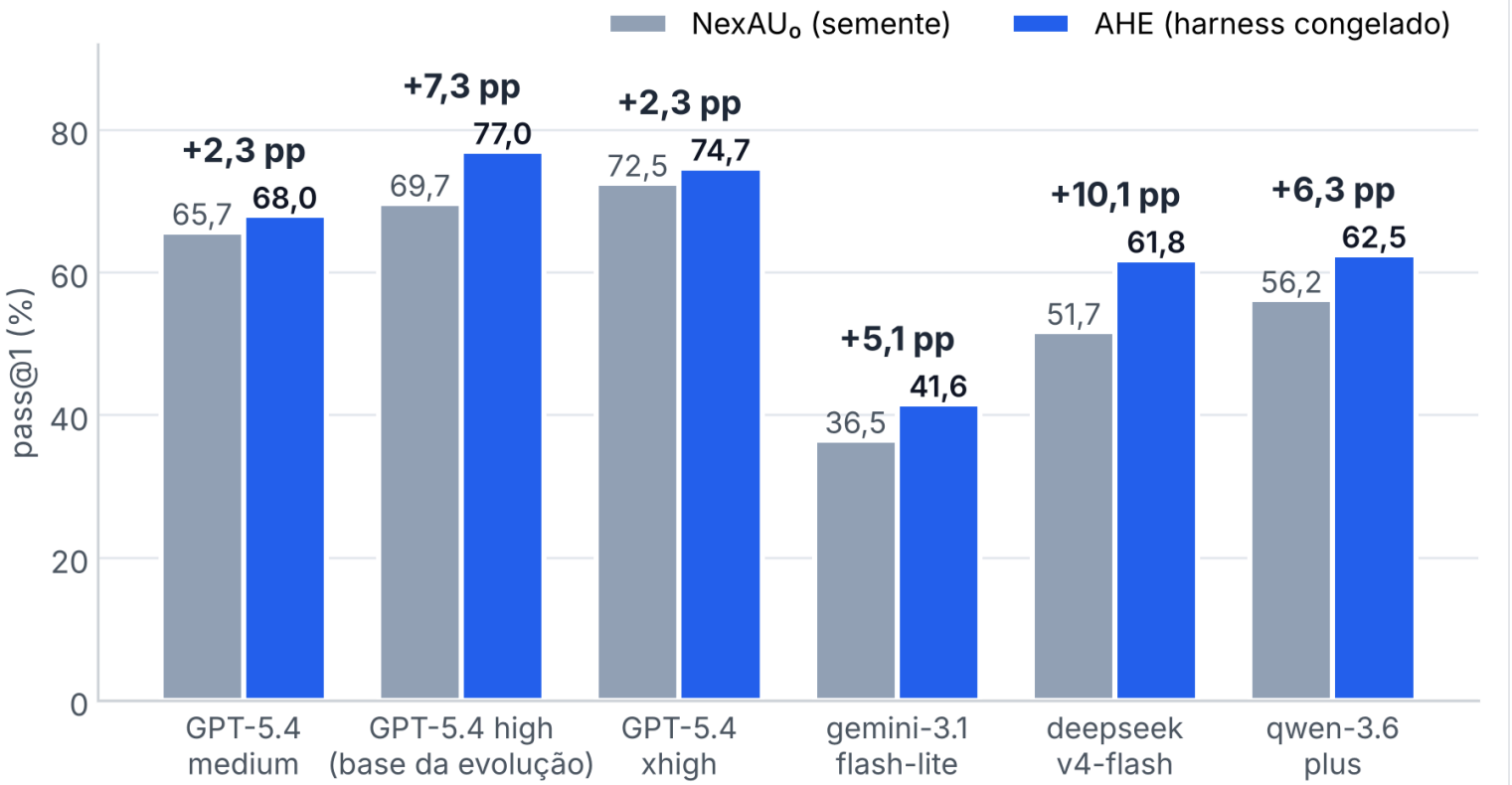
O ganho aqui é sobretudo de custo: mesmo desempenho com menos tokens.

Fonte: Lin et al. (2026), Tabela 2 e Seção 4.3.

13 RQ2 (2/2): transferência entre modelos-base

PERGUNTA (RQ2)

O AHE sofre overfitting ao alvo da otimização? Aqui: transfere para **outros modelos-base**?



Ganhos de +2,3 a +10,1 pp

- Em cinco outras bases, **sem nova evolução** (GPT-5.4 high é a base da evolução).
- Famílias diferentes ganham mais: deepseek +10,1; qwen +6,3; gemini +5,1.

Ressalva dos autores

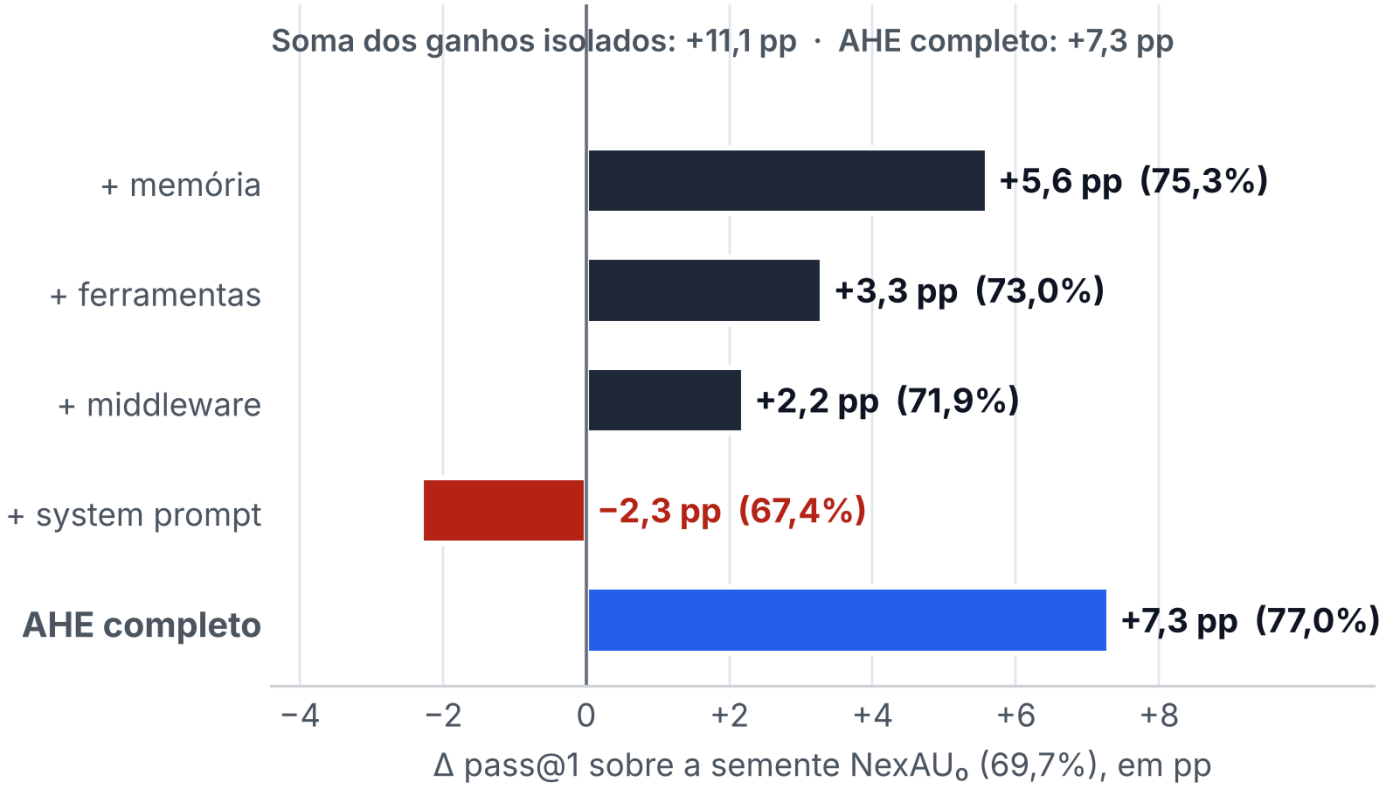
- Orçamento de passos e timeout ajustados ao GPT-5.4 high.
- Perfil não monotônico no GPT-5.4: +2,3, +7,3, +2,3.

Fonte: Lin et al. (2026), Figura 3 e Seção 4.3 (valores lidos da figura).

14 RQ3a: onde está o ganho? Ablação por componente

PERGUNTA (RQ3A)

O que dentro do AHE gera os ganhos? Em quais componentes o valor se acumula?



Análise

- **Ganhos isolados:** memória +5,6; ferramentas +3,3; middleware +2,2 pp.
- **Prompt sozinho:** −2,3 pp (única regressão).
- **Não aditivo:** soma dos positivos = +11,1 pp; AHE completo = +7,3 pp.

Nível Difícil

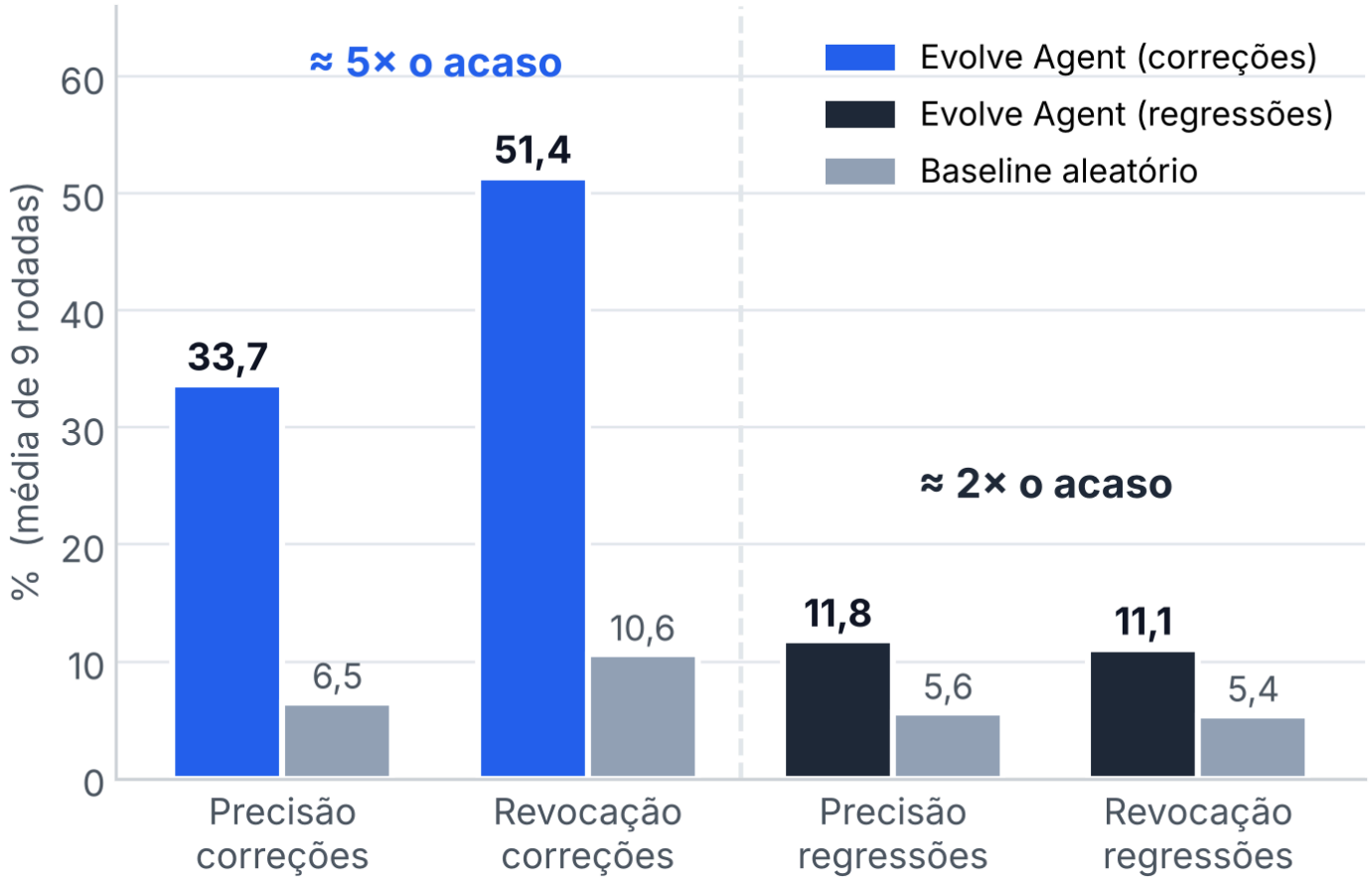
Só a memória (**63,3%**) supera o AHE completo (**53,3%**): empilhar componentes gasta turnos em rechecagens redundantes de fechamento.

Fonte: Lin et al. (2026), Tabela 3 e Seção 4.4.1 (Terminal-Bench 2, 89 tarefas).

15 RQ3b: a autoatribuição é confiável?

PERGUNTA (RQ3B)

As previsões de correção e de regressão do Evolve Agent batem com a realidade?



≈ 5×
acima do acaso nas **correções**

≈ 2×
acima do acaso nas **regressões**

Análise

- O alvo de cada edição é real; o agente **não vê o que vai quebrar**.
- Em 9 rodadas: 43 regressões previstas, 5 acertaram; 40 ocorreram sem serem previstas.

Trabalho futuro apontado pelos autores: prever regressões.

Fonte: Lin et al. (2026), Figura 4, Seção 4.4.2 e Apêndice D.

16 Análise crítica

1 Evolução e avaliação no mesmo conjunto

Evolui e reporta nas mesmas 89 tarefas do Terminal-Bench 2. O único teste fora da amostra é o SWE-bench.

2 Ganho pequeno fora do benchmark

SWE-bench: +0,4 pp em sucesso (75,2% para 75,6%). O benefício é de custo (−12% de tokens).

3 Sem robustez a operação e modelo

Ajustado ao GPT-5.4 high; três papéis com o mesmo modelo; uma única rodada de experimento.

4 Regressões imprevisíveis

Precisão de ~11,8% ao prever regressões; componentes não se somam e o perfil é não monotônico.

5 Limites declarados

Só 2 benchmarks; sem outras linguagens, repositórios grandes ou humano no laço; “protótipo de pesquisa”.

Veredito: proposta de engenharia bem pensada, com contrato falsificável e rollback; evidência sugestiva, não conclusiva.

Fonte: análise do apresentador com base em Lin et al. (2026), Tabelas 1 a 3, Figuras 1 e 4, Limitations.

17 Conclusões

69,7% → 77,0%
pass@1 no Terminal-Bench 2 em 10 iterações

−12%
tokens no SWE-bench, com sucesso estável

+2,3 a +10,1 pp
em cinco outras bases, sem reevolução

5× vs. 2×
acerto acima do acaso: correções vs. regressões

1. **Superfície de adaptação**
   Harness com componentes como arquivos e edições como commits.

2. **Observabilidade**
   Torna a evolução autônoma auditável e reversível.

3. **Onde está o ganho**
   Ferramentas, middleware e memória, **não no prompt**.

4. **Transferência**
   Harness congelado transfere entre modelos; no SWE-bench, mais barato.

Visão dos autores e ligação com Zhong & Zhu

A evolução em nível de harness é um **eixo complementar ao treino do modelo**: uma superfície externa e auditável onde a experiência do agente se acumula.

Zhong & Zhu (2026) dão o vocabulário do harness; o AHE traz um primeiro teste empírico automatizado.

Obrigado! Perguntas?

Fonte: Lin et al. (2026), Seção 5; Zhong & Zhu (2026).

18 Referências

- LIN, J. et al. Agentic Harness Engineering: Observability-Driven Automatic Evolution of Coding-Agent Harnesses. arXiv:2604.25850v4, 2026.
- ZHONG, H.; ZHU, S. AI Harness Engineering: A Runtime Substrate for Foundation-Model Software Agents. arXiv:2605.13357v1, 2026.
- NEX-AGI. NexAU: a general-purpose agent framework. github.com/nex-agi/NexAU, 2025.
- NEX-AGI TEAM et al. Nex-N1: Agentic models trained via a unified ecosystem for large-scale environment construction. arXiv:2512.04987, 2025.
- MERRILL, M. A. et al. Terminal-Bench: Benchmarking agents on hard, realistic tasks in command line interfaces. arXiv:2601.11868, 2026.
- JIMENEZ, C. E. et al. SWE-bench: Can language models resolve real-world GitHub issues? ICLR, 2024.
- ZHANG, Q. et al. Agentic context engineering: Evolving contexts for self-improving language models (ACE). ICLR, 2026.
- CAI, Y. et al. Training-free group relative policy optimization. arXiv:2510.08191, 2025.
- LIN, L. Agent Debugger: Understanding agent trajectory with agentic workflows. dawning-road.github.io/blog/agent-debugger, 2026.
- OPENAI. Codex CLI, 2025; Introducing GPT-5.4, 2026. ANOMALY. OpenCode, 2025. HARBOR. Terminus-2, 2026.